DSL Compiler Documentation

1 General Explanation

The DSL compiler is a parser and semantic analyzer for the **Domain-Specific Language for Football**. It reads source code files written in DSL, verifies their syntax and semantic correctness, and outputs a message in case of errors (either syntactic or semantic). If the input is correct, it prints a success message and produces a JSON file with information about teams and matches.

This JSON output can then be used in other software for match analysis or visualization.

2 Grammar of the Language

The DSL grammar defines the structure of valid programs. It includes declarations of teams, players, and matches with their respective attributes. Functions such as `set` and `has` are also supported to modify or query attributes dynamically.

3 Input Format

The input is a plain text file with valid DSL source code that should include:

- Team declarations before their use in matches.
- Players with valid roles and unique numbers within the same team.
- Matches referencing only previously declared teams.
- Use of functions to modify or check teams, players, and matches.

Example:

```
Team "Juventus" {
    Player "Buffon" position "GK";
    Player "Chiellini" position "CB";
    ...
    Formation 4-3-3;
    Coach = "Allegri";
    Captain = "Buffon";
    Vicecaptain = "Chiellini";
    Strategy = "HighPress";
}
Match "Day 1 First Match" {
    Stadium = "Allianz Stadium";
    HomeTeam = "Juventus";
    AwayTeam = "Inter";
    ...
}
hasStadium("Day 1 First Match", "Allianz Stadium");
setStadium("Day 1 First Match", "San Siro");
```

4 Semantic Checks

The compiler enforces the following semantic constraints:

- **Unique Identifiers**: Player names must be unique within a team; team names must be globally unique.
- **Valid Player Roles**: Each role must match predefined categories.
- **Predefined Roles**:
  - `GK`
  - Defenders: `CB, RB, LB, SB, WB`
  - Midfielders: `CM, CD, AM, DM, RM, LM`
  - Attackers: `ST, CF, RW, LW, RF, LF`
- **Team Existence**: Matches must reference already declared teams.
- **Player Existence**: Players must be declared in teams; empty teams are not allowed.
- **Date and Score Format**: Dates must be valid ISO (YYYY-MM-DD); scores must be in `int-int` format.
- **Scorers Validity**: Each scorer must belong to a participating team.
- **Formation Constraints**: Teams must contain exactly one goalkeeper, and a number of players between 8 and 11. The declared formation must be respected.

5 Output Format

If parsing and semantic validation succeed, the compiler outputs a JSON object structured as follows:

```
{
  "teams": [
    {
      "name": "Inter",
      "coach": "Inzaghi",
      "captain": "Handanovic",
      "viceCaptain": "Barella",
      "strategy": "CounterAttack",
      "formation": "3-5-2",
      "players": [
        { "name": "Handanovic", "position": "GK" },
        ...
      ]
    },
    {
      "name": "Juventus",
      ...
    }
  ],
  "matches": [
    {
      "name": "Day 1 First Match",
      "stadium": "San Siro",
      "homeTeam": "Juventus",
      "awayTeam": "Inter",
      "date": "2023-11-06",
```

```
      "time": "18:00",
      "weather": "Sunny",
      "result": "1-1",
      "referee": "Rocchi",
      "scorers": [ "Vlahovic", "Lukaku" ]
    }
  ]
}
```

6 How to Run the Compiler

Prerequisites

- C compiler (e.g., `gcc`)
- `flex`, `bison`

Compilation Steps

```
bison -d -v dls-bison.y
flex dls-lex.l
gcc -o parser lex.yy.c dls-bison.tab.c progetto.c -lfl
```

Usage

```
./parser < input.txt
```